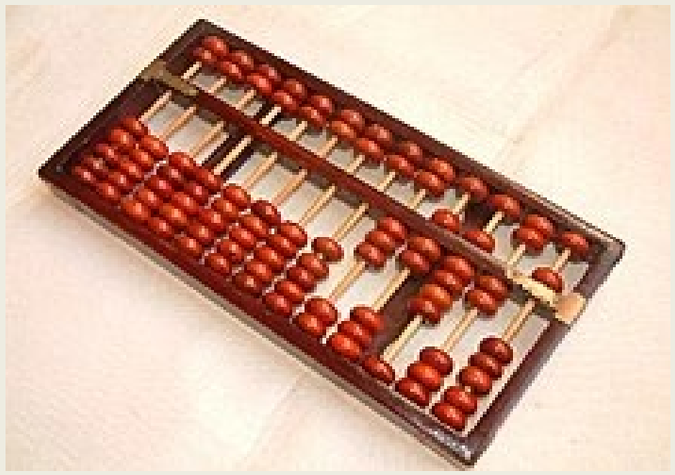
Тема 1. Разработка на програма. Компоненти на програмния текст

Съдържание

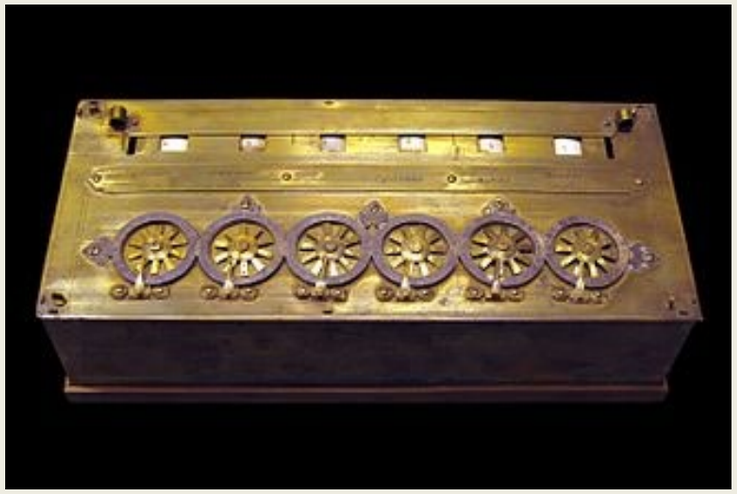
- История на компютрите
- Разлика между системен и приложен софтуер
- Анализ на стъпките за разработка на софтуер
- История на C++
- Компоненти на програмния текст
- Азбука на езика C++
- Лексеми (tokens)
- Служебни думи
- Имена (идентификатори)
- Литерали
- Числа (числени константи)
- Символни константи
- Празни символи
- Команда (оператор/statement)
- Блок
- Директиви
- Коментари
- Използване на namespace std
- Оператори

История на компютрите

- Компютрите датират от преди 5000 години
- Съвременни компютри са четвърто или пето поколение
- Предмодерни компютри
  - **Сметало** (**Абак**)

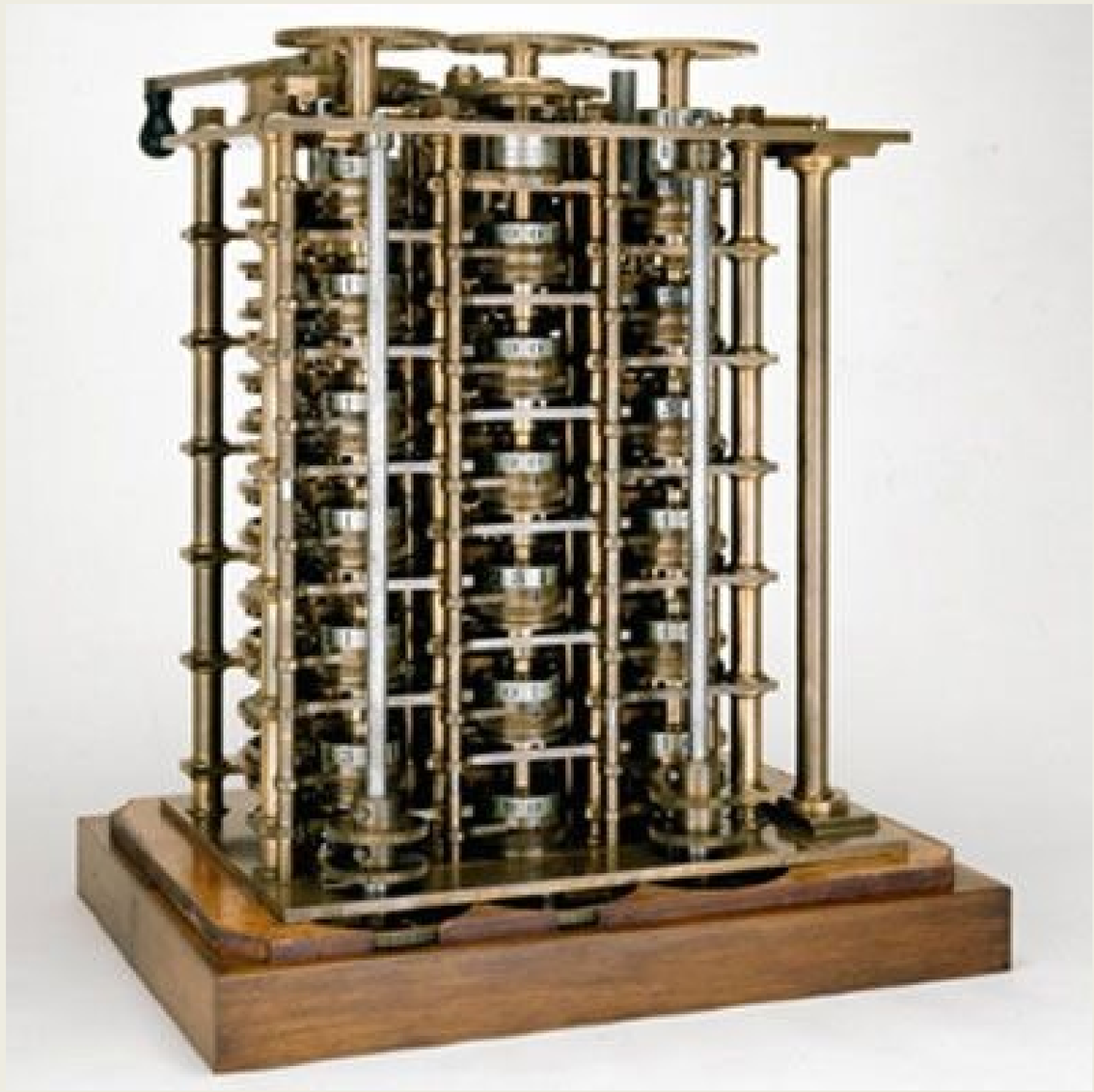


  - Сметачна машина (Калколатор на Паскал или Паскалин, 1642)



История на компютрите

- Предмодерни компютри
  - Аналитична машина (Analytical Engine, Чарлз Бабидж и Ада Лъвлейс, 1830 г.



История на компютрите

- Предмодерни компютри
  - **Първо поколение**, отличаващо се с употребата на **вакуумни тръби** (средата на 1940 г.)



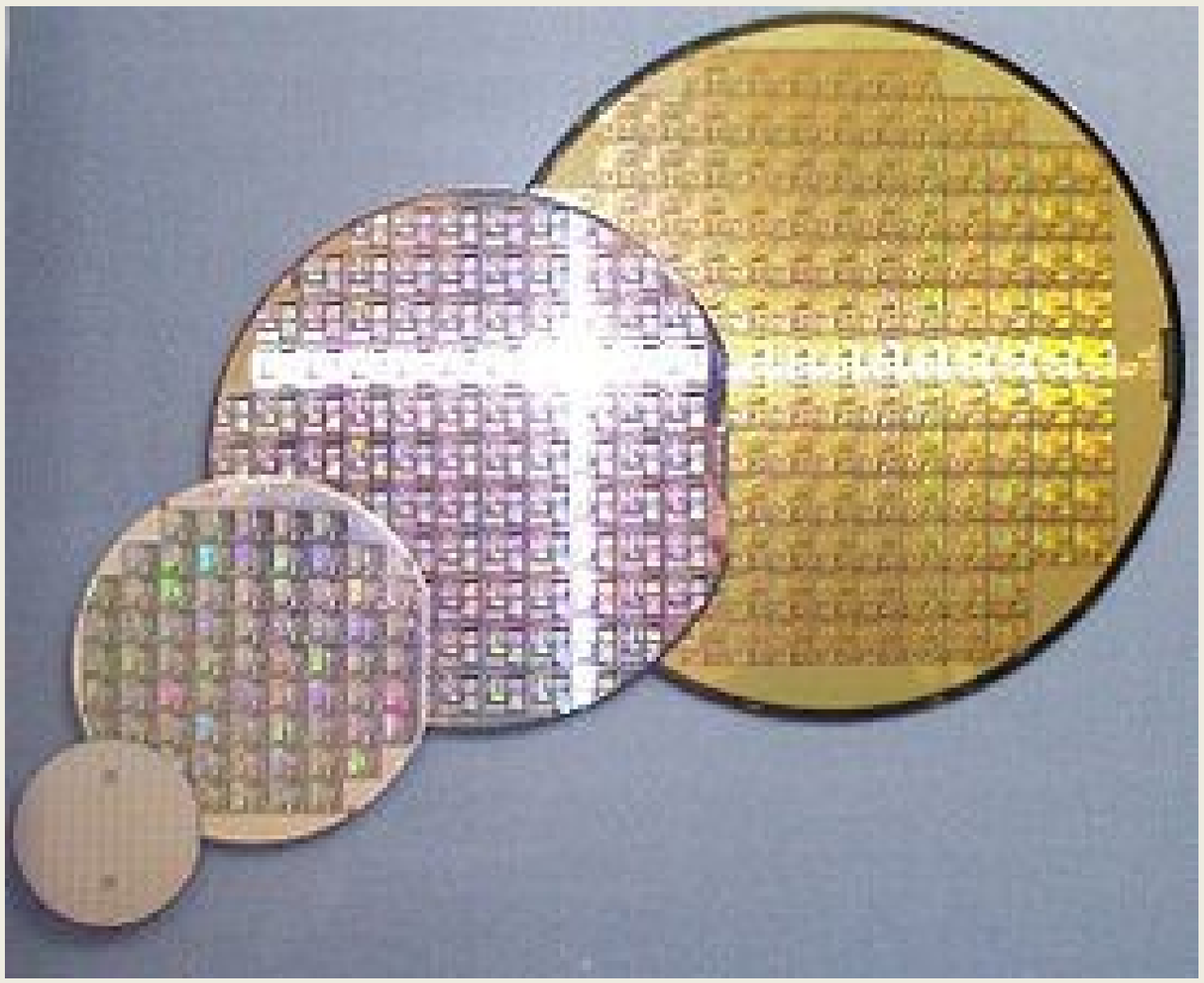
История на компютрите

- Предмодерни компютри
  - **Второ поколение**, отличаващо се с употребата на транзистори (средата на 50-те години)



История на компютрите

- Предмодерни компютри
  - **Трето поколение** – транзистори, притиснати в малки силиконови дискове (1964-1971)
    - Компютрите стават по-малки
    - За първи път се виждат операционни системи



История на компютрите

- Предмодерни компютри
  - **Четвърто поколение** – производителите на компютри предоставят компютрите на обикновените потребители
    - IBM представя първия персонален компютър (PC) (1981)



История на компютрите

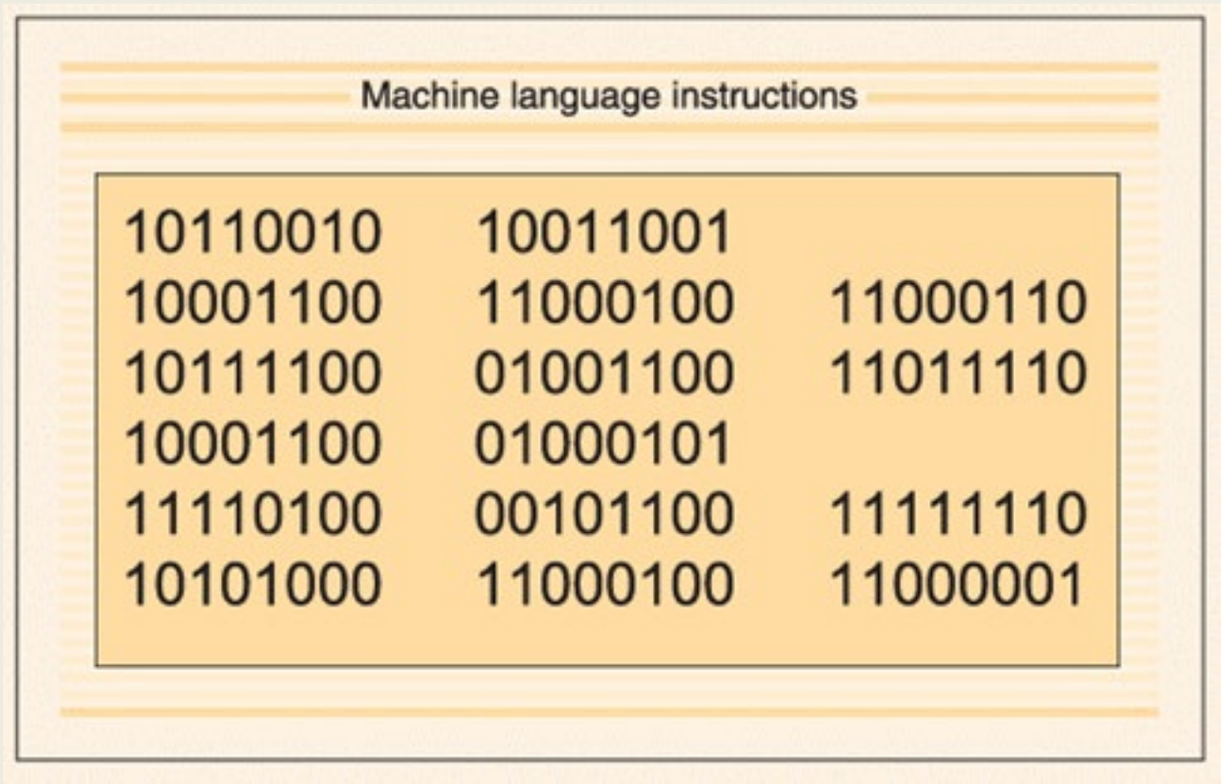
- Предмодерни компютри
  - **Пето поколение** – по-трудно за дефиниране
    - Компютрите приемат устни инструкции
    - Компютрите имитират човешки разсъждения чрез AI
    - Компютрите комуникират глобално
    - Мобилните и безжичните приложения се разрастват

Системен и приложен софтуер

- Софтуерът се състои от **програми**
  - **Множество от инструкции**, които казват на компютъра какво точно да направи и водят до определен изходен резултат
  - Програмните инструкции обикновено се изпълняват от **централния процесор.**
- Два вида софтуер
  - **Системен**
  - **Приложен**
- В някои случаи, за да работи коректно една компютърна програма, тя се нуждае от **входни данни**, които се подават от **потребителя** или се взимат от **друг източник** (бази от данни, файлове, Интернет, и др.).
- **Силата** на това, което компютърът прави, зависи от това какви типове софтуер са налични

Системен софтуер

- Системният софтуер **е повече от** операционни системи
- **Операционна система**
  - Зарежда се при включване на компютъра
  - Примерите включват **Windows**, **Linux**, **Android**, iOS
  - Включва помощни програми за файлова система, комуникационен софтуер
- Включва **компилатори**, **интерпретатори** и **асемблери**



Приложен софтуер

- **Приложният софтуер** изпълнява специфична задача
  - Текстообработващи програми, електронни таблици, заплати, инвентар

- **Създава се** чрез език за програмиране от високо ниво
  - **C#**, Java, Visual Basic, C++

- **Компилатор**
  - Превежда инструкциите в машинно четима форма
  - Първо проверява за нарушения на правилата
    - Синтактични правила – как се пишат изрази

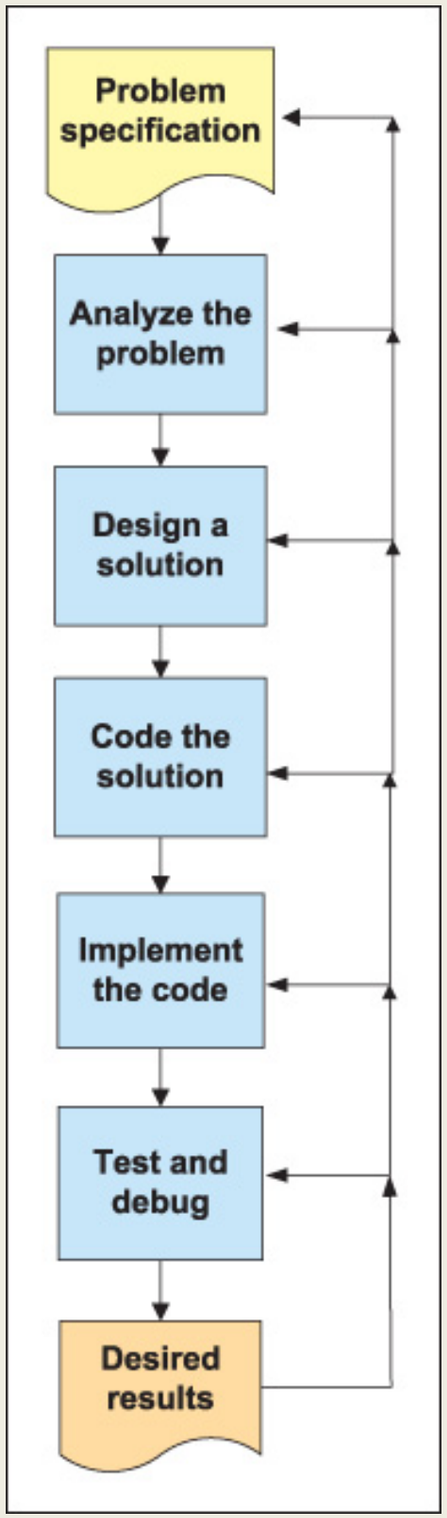
Стъпки в процеса на разработване на програма

1. Анализиране на проблема
2. Проектиране на решение
3. Кодиране на решението
4. Внедряване на кода
5. Тестване и отстраняване на грешки

Стъпки в процеса на разработване на програма

- Процесът на разработка на софтуер е **итеративен процес**

- Когато се открият грешки, често е необходимо да се върнете към предишна фаза или стъпка



Програмиране

- Една компютърна система се състои от **хардуер** и **софтуер**;
- Компютрите са изградени от хардуерни елементи, а управлението и работата им зависят от т.нар. софтуер;
- **Софтуерът е съвкупност от команди** или т.нар. програми, които са създадени от хора нар. програмисти;
- За създаването на софтуер в различните направления се използват различни компютърни езици - C, **C++**, C#, Java, Python, JavaScript, SQL, PHP, HTML, CSS, Scala, R и много др.
- Обект на нашият курс ще е създаването на софтуер с езика C++.

Компютърна информатика

Съществуват различни направления в компютърната информатика:

- **Връзка компютър-потребител** - графика, интелигентни интерфейси, входно-изходни технологии
- **Компютърни комуникации и сигурност** - сигурност, компютърни мрежи, разпределени агенти
- **Новаторски компютърни архитектури** - асинхронни архитектури, висока ефективност
- **Моделиране на сложни феномени** - механика, биология, икономика, динамика на флуидите
- **Нетрадиционни подходи за изчисление** - невронни мрежи, генетични алгоритми, квантови изчисления
- **Изкуствен интелект** - моделиране на разумно мислене, моделиране на системи за заместване на човека

История на езика на c++

- Езикът C се създава и развива заедно с операционната система UNIX.
  - *Първата версия е създадена през 1973 г. от Денис Ричи.*
  - *Езикът се развива интензивно и през 1977 г. UNIX е пренаписана на C.*
  - *Първоначално е създаден и имплементиран единствено за операционни системи, а по-късно се откриват други приложения на езика*
- През 1983 American National Standards Institute (ANSI) създава комитет с цел създаване на еднозначна дефиниция на езика C
- През 1988 излиза ANSI стандартът за C, наричан още “ANSI C”
- Стандартна версия на езика е установена през 1989 година.

История на езика на c++

- C++ (произнася се „си-плюс-плюс“) е не специализиран **език за програмиране** от високо ниво. Той е **обектно-ориентиран** език със статични типове.
- От 1990-те, C++ е един от най-популярните комерсиални езици за програмиране.
- **Бьорн Строуструп (Bjarne Stroustrup)** разработва C++ през 1983 г. в **Лабораториите Бел** като разширение на **езика C** – езикът е базиран на C, но в него са добавени редица допълнителни възможности и са направени няколко промени.
  - Нов обектно-ориентиран език, който веднага получава много широко разпространение.
  - Стандартизацията на езика завършва през 1998 година, като книгата на Страуструп е всъщност този стандарт.
- Основната разлика между C и C++ е, че C++ съдържа вградена в езика поддръжка на обектно-ориентирано програмиране.
- В C++ са добавени класове, множествено наследяване, виртуални функции, припокриване (overloading), шаблони (templates), обработка на изключения (exceptions) и вградени оператори за работа с динамична памет.

История на езика на c++

- Езиковият стандарт на C++ е ратифициран през 1998 като ISO/IEC 14882:1998, през 2003 година има преразглеждане на стандарта – ISO/IEC 14882:2003, а последната версия е от декември 2020 – *ISO/IEC 14882:2020* (известна като C++20). Следващата планирана версия е C++23.
- Повечето програми на C могат директно или със съвсем малки модификации да бъдат компилирани с компилатор за езика C++.
- Поради паралелното си развитие с Turbo Pascal и Object Pascal в посока обектна ориентираност, C++ има общи черти с тях, но съществуват и редица разлики.
- Поради заимстване на множество концепции от C++, езикът Java има много общи черти със C++.
- Една голяма част от приложните програми на много операционни системи, както и някои от самите операционни системи, са написани на този език.

Характеристики на езика C++

- Популярните езици C++ и Objective-C са разширения на езика C.
- C++ е език за програмиране от средно ниво с общо предназначение
- Ниско ниво на абстракция
- Голям контрол върху хардуера
- Бързодействие, пряк достъп до паметта и бърз контрол върху данните
- Подходящ за системно програмиране, както за операционни системи, така и за различни приложения
- Неподходящ за web-приложения
- Лесно преносим на различни платформи

Среди за програмиране
Integrated Development Environment (IDE)

- Всички **C++** програми първоначално се запаметяват като обикновен текстов файл с разширение **.cpp**.
  - *Може да създавате **C++** програми чрез обикновен текстов редактор, какъвто е **Notepad** в **Windows** или **EMACS** в **Linux**.*
- Преди да се изпълни една **C++** програма компилаторът я превръща в байт код, който може да бъде разбран от компютъра. И накрая програмата се превежда в *машинно-четим и изпълним формат*, т.е. **.exe** файл.
- За да компилирате една програма можете да използвате компилатор с визуално интегрирана среда.
  - *Microsoft Visual Studio 2019 / 2022 (https://visualstudio.microsoft.com/vs/)*
  - ***Code::Blocks 20 (http://www.codeblocks.org/downloads/binaries/)***
  - *Dev-C++*
  - *NetBeans*

Компютърна програма на c++

Сорс код

```
1.  //my first program in C++
2.  #include <iostream>
3.
4.  int main()
5.  {
6.      std::cout << "Hello world!" << endl;
7.      return 0;
8.  }
```

Изход

```
Hello World!
```

Компоненти на програмния текст

Нека прегледаме кода на програмата ред по ред:

- **Ред 1**: `// my first program in C++`
- **Две наклонени черти** показват, че останалата част от реда е **коментар**, въведен от програмиста, но те нямат ефект върху поведението на програмата. Програмистите ги използват, за да слагат кратки пояснения или наблюдения, които засягат кода или програмата.

Компоненти на програмния текст

- **Ред 2**: #include **<iostream>**
  - *Редове, които започват със **знак диез (#) са директиви** прочетени и интерпретирани от т.нар. препроцесор.*
  - *Специални редове, които се интерпретират **преди дори да започне компилацията на програмата**.*

  *#include **<iostream>** - инструктира препроцесора да включи част от **стандартния C++ код**, познат като заглавен файл **iostream** (header), това ни позволява да използваме стандартни операции за вход и изход, като записа на изхода от нашата програма (**Hello World**) на екрана.*

- **Ред 3**: Празен ред - **не указват ефект върху програмата**. Подобряват четимостта на кода.

Компоненти на програмния текст

- Ред 4: int main()
  - *Този ред показва **декларирането на функция**.*
  - *По същество функцията е група оператори, на които е дадено име, т.е. името "main" на групата изрази, които следват.*
- Функцията с име **main** е специална функция във всички C++ програми; това е функцията, която се извиква, когато програмата се изпълнява. Наричат я още **главна функция**.
- Изпълнението на всички C++ програми започва с главната функция, независимо къде функцията е действително разположена в кода.

Компоненти на програмния текст

- **Ред 5 и 8**: { и }
- **Отварящата фигурна скоба**({) на ред 7 показва началото на дефиницията на главната функция **main**, а **затваряща фигурна скоба** (}) на ред 8, определя нейния край.
- Всичко между тези скоби е **тялото на функцията**, която определя какво ще се случва, когато функцията **main** се извика.
- Всички функции използват скоби, за да покажат началото и края на техните дефиниции.
- По принцип набор от инструкции заградени във { } се нарича блок.

Компоненти на програмния текст

- **Ред 6**: `std::cout << "Hello world!" << endl;`
  - C++ израз (оператор) – Един оператор е израз, който в действителност допринася за ефект или действие на програмата. Той е в основата на една програма, като специфицира нейното поведение.
  - Изразите се изпълняват на същия ред, в който се появяват в рамките на тялото на функцията.
- Този израз има три части:
  - **Първо, `std::cout`**, който идентифицира стандартното изходно устройство (обикновено това е екранът на компютъра).
  - **Второ, оператора за вмъкване (<<)**, който показва, че това, което следва, се вмъква в std :: cout.
  - **И накрая, едно изречение в кавички ("Hello world!")** е съдържанието, показано в стандартния изход.

Компоненти на програмния текст

- Забележете, че изразът завършва с **точка и запетая (;)**. Този разделител определя края на израза, както точката завършва изречението в човешкия език.
- Всички C++ изрази трябва да завършват със знака за точка и запетая.

!!! Една от най-често допусканите синтактични грешки в C++ е забравянето да се завърши израз с точка и запетая.

Компютърна програма на c++

Сорс код

```
// my second program in C++
#include <iostream>

int main (){
  std::cout << "Welcome to New Bulgarian University World! ";
  std::cout << "I'm a happy C++ programmer";
}
```

Изход

```
Welcome to New Bulgarian University World! I'm a happy C++ programmer
```

Азбука на езика C++

Азбуката на езика включва:

- **Букви**: главните и малките букви от латиницата, като се прави разлика между главна и малка буква;
- **Цифри**: 0, 1, . . . ,9;
- **Специални символи**:

~ ! @ # $ % ^ & * ( ) - + | { } [ ] . , ? : ; " ' < > / = _

Лексеми (tokens)

- От азбуката на езика по определени правила се образуват **думите на езика**, които се наричат лексеми.
- Лексемите в C++ са няколко вида:
  - **ключови**/служебни думи;
  - **имена** (идентификатори);
  - **числени константи** (числа);
  - **символни константи**;
  - **символни низове** (стрингове);
  - **оператори**;
  - **разделители** (препинателни знаци/delimiters).

Служебни думи

- **Ключовите думи** са резервирани думи в езика.
- Чрез тях се означават различни оператори, модификатори, декларации и др.
- Ключовите думи на езика C++ са дефинирани в стандарта ANSI, но конкретните компилатори поддържат и ключови думи, които са извън този стандарт.

`auto`	`goto`	`typedef`	`explicit`	`static_cast`
`double`	`do`	`union`	`false`	`template`
`break`	`if`	`unsigned`	`friend`	`this`
`else`	`int`	`void`	`inline`	`throw`
`case`	`long`	`volatile`	`mutable`	`true`
`enum`	`register`	`while`	`namespace`	`try`
`char`	`return`	`asm`	`new`	`typeid`
`extern`	`short`	`bool`	`operator`	`typename`
`const`	`signed`	`catch`	`private`	`using`
`float`	`sizeof`	`class`	`protected`	`virtual`
`continue`	`static`	`const_cast`	`public`	`wchar_t`
`for`	`struct`	`delete`	`reinterpret_cast`	
`default`	`switch`	`dynamic_cast`		

Имена (Идентификатори)

- На компонентите на програмите (променливи, константи, типове, класове, обекти, функции и др.) се дават **имена** (идентификатори).
  - Идентификаторите могат да съдържат **букви от латиницата** (малки и големи), **цифри** (от 0 до 9) и символа **долна черта** (_), като първият символ не трябва да бъде цифра.
- Ето някои примери на правилно и неправилно конструирани идентификатори:
  - aBCd
  - A34
  - _identifier
  - _id2_
  - 2abs //ГРЕШКА! Първият символ не може да бъде цифра
  - _id#23 //ГРЕШКА! Символът # е недопустим
- В съвременните компилатори максималната **дължина на идентификаторите** може да се задава, като подразбиращата се обикновено е **32 символа**.

Литерали

- **Символно представяне на конкретни стойности** на числа, текст, букви и др., например 12, 23.67, "TEXT", 'd'.
- При компилация литералите се преобразуват във вътрешно (двоично) представяне на съответните стойности;
- **Текстовите** литерали представляват ограден с кавички текст. Пример: "Това е литерал от символи".
- **Символните** литерали представляват заградени в апострофи единични печатни символи, например 'F'.

Литерали

- Съществуват и **непечатни символи, използвани служебно**, напр. за звуков сигнал, за управление на печата и др.
- Тези символи се представят в текста на програмата като **‘\’** (обратна наклонена черта, backslash), **последвана от други символи**.

Означение	Смисъл
\b	Връщане с един символ назад
\n	Преминаване на нов ред
\r	Връщане в началото на реда
\t	Табулация
\v	Вертикална табулация
\"	Кавичка
\'	Апостроф
\\	Обратна наклонена черта
\f	Преминаване на нова страниа
\a	Звуков сигнал
\?	Въпросителна
\N	Символ с ASCII код, записан в осмична бройна система
\xN	Символ с ASCII код, записан в шестнадесетична бройна система

Компютърна програма на c++

Сорс код

```
/*
   my second program in C++
   with more than one lines
*/

#include <iostream>

int main ()
{
  std::cout << "Hello World! \n";
  // prints Hello World!
  std::cout << "I'm a \"C++ program\"";
  // prints I'm a "C++ program" in a new line
}
```

Изход

```
Hello World!
I'm a "C++ program
```

Числа (Числени константи)

- **Числата** в програмите (положителни и отрицателни) могат да бъдат цели и дробни.
- **Целите числа** могат да бъдат задавани в десетична, шестнадесетична и осмична бройна система.
- **Шестнадесетичните числа** започват с 0x, а **осмичните** с 0, както е показано в следните примери:
  - 2345 - десетично положително число 2345
  - -32 - десетично отрицателно число -32
  - 0xAF - шестнадесетично число AF (175 десетично)
  - 023 - осмично число 23 (3*1 + 2*8 = 19 десетично)
- **Дробните числа** могат да се записват по два начина: във формат с фиксирана точка (F - формат) и в експоненциален формат (E - формат), например: 3.45, 6.7e-2.

Символни константи & Празни символи

- Символните низове са лексеми, които се състоят от **множество символи, оградени от двойни кавички**, например:
  - "Това е низ"
  - "03 това също е низ"
- Елементите на символните низове се подреждат последователно в паметта, като всеки елемент заема по един байт.
- Последният байт на всеки символен низ съдържа 0 (символната константа), която се добавя автоматично от компилатора и е индикатор за **край на низа**. Следователно низ, който съдържа **n** символа, заема **n+1** байта в паметта.
- Между две съседни в текста лексеми може да се поставят произволен брой **празни символи** (white spaces) - интервали, знаци за табулация, край на ред, като това не променя смисъла на програмния текст.
  - Постигане на удобен за четене вид на текста.

Израз (оператор/statement)

- Съставена според правилата **последователност от лексеми**, завършваща с разделителя ‘**;**’ представлява **израз** или **оператор** (**statement**) от програмата.
- Ако продължим аналогията с текст на естествен език, **операторът е аналог на изречение**.
  - *Това се потвърждава и от буквалния превод на „**statement**” – твърдение, изявление.*
  - *Другият възможен превод на **statement** е „инструкция”.*
- По традиция, в терминологията на български „**инструкция**” се употребява при програми на асемблер или в машинен код, а „**оператор**” – при програми на езици от високо ниво.
  - *Известно объркване внася директното използване в професионалния жаргон на английския термин „**operator**”, който означава „знак за операция” като +, * и пр.*
- В лекциите винаги ще се съблюдава следното съответствие: „**оператор**” = „**statement**”, „**знак за операция**” =„**operator**”.

Блок

- Блок се нарича всяка последователност от оператори, ограничена с фигурни скоби ‘{‘ и ‘}’.

```
{
    std::cout << "Hello World! “;
    std::cout << "I'm a C++ program“;
}
```

- Блоковете могат да се влагат един в друг.

```
{
    std::cout << "Hello World! “;
    {
        std::cout << “Nice to meet you! “;
        std::cout << "I'm a C++ program“;
    }
    std::cout << "I'm a C++ future developer“;
}
```

Директиви

- В програмния текст на C/C++ присъстват и директиви, които не са част от програмата, а управляват нейната обработка: заместване на текст, компилиране и свързване.
  - *Директивите **започват със знака ‘#’** и винаги са на отделен ред.*
- Директивите за предварителна обработка контролират поведението на препроцесора.
  - *Инструкция за предварителна обработка (една от следните: **define, undef, include, if, ifdef, ifndef, else, elif, endif, line, error, pragma**)*
  - *Аргументи (зависят от инструкциите)*
  - *Важат до края на реда*
- Позволена е и празна директива (# следвана от нов ред), която няма никакъв ефект върху кода.

Директиви - възможности

- Препроцесорът има възможностите за превод на сорс кода на изходния файл:
  - ***условно*** *компилиране на части от сорс кода (контролирани от директиви като* ***#if, #ifdef, #ifndef, #else, #elif и #endif****).*
  - ***заменя*** *текстови макроси, които евентуално свързват или цитират идентификатори (контролирани от директиви* ***#define*** *и* ***#undef****, и оператори* ***#*** *и* ***##)***
  - ***включване*** *на други файлове (контролирани от директиви* ***#include*** *и проверени чрез* ***__has_include****)*
  - *Възникване на* ***грешка*** *(контролирани от директива* ***#error****)*
- Следните аспекти на препроцесора могат да бъдат контролирани:
  - ***поведение дефинирано чрез имплементация*** *(контролирано от директива #pragma и оператор _Pragma)*
  - ***информация за име на файл или номер на ред*** *достъпна за препроцесора (контролирано от директива #line)*

Коментари

- Навсякъде в текста, където е допустимо да присъстват празни символи, може да се вмъкне **коментар.**
- Коментарът е текст, който **не е част от програмата**, а пояснение за човека, четящ програмния текст.
  - Коментарът може да бъде на **множество редове** - ограничен с “ /*” и “*/”

```
/*
   comment in C++
   with more than one lines
*/
```

  - В C++ се поддържа и коментар **до края на реда** предшестван от знаците “//”.

```
// line comment
```

Използване на *namespace* *std*

- Ако сте виждали код на C++ и преди, може да сте срещали използване директно на ***cout*** вместо ***std::cout***.
  - *И двата израза се обръщат към един и същ обект:*
  - ***cout*** *използва неквалифицирано име (*`cout`*),*
  - ***std::cout*** *квалифицира името директно чрез именованото пространство* `std` *(като* `std::cout`*).*
- ***cout*** е част от стандартна библиотека и всички елементи, които са в стандартната C++ библиотека са декларирани в т.нар. именувано пространство (*namespace)* **std**.
- За да се **използват елементи от** именуваното пространство **std**, една програма или трябва да квалифицира всяка една употреба на елементи от библиотеката (както направихме по-горе), или да се въведе видимост на компонентите ѝ.
  - *Най-типичният начин за въвеждане на видимост на компоненти е чрез следната декларация:* ***using namespace*** **std;**
  - *Достъпа до всички елементи декларирани в* `std` *ще бъде по неквалифициран начин (без префикс* ***std::****).*

Компютърна програма на c++

Сорс код

```
// my third program in C++
#include <iostream>

using namespace std;

int main (){
    cout << "Hello World! ";
    cout << "I'm a C++ program";";
}
```

Изход

```
Hello World! I'm a C++ program
```

Сорс код

```
// my fourth program in C++
#include <iostream>

using std::cout;

int main (){
    cout << "Hello World! ";
    cout << "I'm a C++ program";
}
```

Изход

```
Hello World! I'm a C++ program
```

- И двата начина за достъп до елементите на именуваното пространство **std** (изрична квалификация и използване на декларации) са валидни в C++ и водят до едно и също поведение.

Оператори

- Операторите (знаците за операции) в C++ имат мнемонични означения.
- В преобладаващата си част те се състоят от по един символ. Такива са операторите **+**, **-**, *****, **/**, **&** и др.
- Някои оператори обаче се означават с два или повече символи (обикновено два), например: **!=**, **>=**, **<=**, **++**, **--**, и др.
- Други оператори се означават чрез ключови думи, например: **sizeof**, **new**, **delete**, **for**, **break**.

Домашна работа

Задача 1. Съставете C++ конзолна програма, чрез която да изведете трите си имена на екрана.

Задача 2. Съставете C++ конзолна програма, чрез която да изведете на екрана, в кой университет учите и по каква програма се обучавате.

Задача 3. Съставете C++ конзолна програма, чрез която да изведете на екрана всички служебни думи на езика C++.

Задача 4. Съставете C++ конзолна програма, чрез която да изведете на екрана какво е значението на понятията лексема, литерал, команда.

Задача 5. Да се проучи за какво служи всяка една директива: define, undef, include, if, ifdef, ifndef, else, elif, endif, line, error, pragma. Можете да ползвате сайта http://www.cplusplus.com

Благодаря за вниманието !!!

Въпроси и коментари